De Podman a Docker Hub

Construir una imagen nueva, etiquetarla con una convención de versiones, autenticarse con un token de acceso y publicarla en **docker.io/guerrero11dan**. Incluye multiarquitectura, firma con Cosign y publicación automatizada desde GitHub Actions.

```
$ podman login docker.io -u guerrero11dan
```

1. Teoría | 2. Cuenta y token | 3. La aplicación | 4. Construir | 5. Etiquetar | 6. Publicar | 7. Verificar | 8. Multiarquitectura | 9. Automatizar | 10. Retos

1. Cómo se nombra una imagen

Antes de publicar hay que entender que el nombre de una imagen no es un nombre: es una dirección completa que Podman resuelve en cuatro partes.

```
docker.io / guerrero11dan / bitacora-api : 1.0.0
─────────   ──────────────   ─────────────   ─────
 registro     namespace      repositorio    tag

registro     dónde vive (docker.io, quay.io, registry.redhat.io)
namespace    tu usuario u organización en ese registro
repositorio  el nombre del proyecto
tag          la versión mutable; debajo hay un digest inmutable
```

Podman siempre quiere el nombre completo. Docker asume `docker.io/library/` cuando escribes `nginx` ; Podman, en cambio, consulta `/etc/containers/registries.conf` y si hay ambigüedad te pregunta cuál registro usar. Es una diferencia de seguridad deliberada: evita que alguien publique una imagen con el mismo nombre corto en un registro que tengas configurado antes en la lista.

Tag contra digest

El tag es una etiqueta móvil: `1.0.0` puede apuntar hoy a un contenido y mañana a otro si alguien lo reescribe. El **digest** (`sha256:…`) es el hash del manifiesto y nunca cambia. En producción se despliega por digest; los tags son para humanos.

Qué viaja realmente en un push

Una imagen es un manifiesto más una lista de capas. Al hacer push, Podman envía solo las capas que el registro aún no tiene: por eso la primera publicación tarda y las siguientes, si solo cambió la última capa, son casi instantáneas. Es la misma razón por la que conviene poner lo que cambia poco (instalación de paquetes) antes de lo que cambia mucho (tu código) en el Containerfile.

Concepto	Docker	Podman
Daemon	dockerd como root	Sin daemon, fork/exec directo
Credenciales	`~/.docker/config.json`	`${XDG_RUNTIME_DIR}/containers/auth.json`
Nombres cortos	Asume Docker Hub	Pregunta o falla según la política
Comando de publicación	`docker push`	`podman push` , misma API OCI

2. Cuenta y token de acceso

Nunca uses tu contraseña de Docker Hub en la terminal. Genera un token de acceso personal, que puedes revocar sin cambiar la contraseña y al que puedes darle permisos limitados.

2.1 Generar el token

1. Entra a hub.docker.com con la cuenta `guerrero11dan` .
2. Menú de tu usuario → **Account settings** → **Personal access tokens**.
3. **Generate new token**. Descripción: *podman-rhel-laboratorio*. Permisos: **Read & Write**.
4. Copia el token ahora; no se vuelve a mostrar.

2.2 Autenticarse desde Podman

terminal Copiar

```
# Guarda el token en una variable sin que quede en el historial
# (el espacio inicial evita el historial si HISTCONTROL=ignorespace)
 read -rsp "Token de Docker Hub: " DH_TOKEN && echo

# Login pasando el token por stdin: no aparece en ps ni en el historial
echo "$DH_TOKEN" | podman login docker.io --username guerrero11dan --password-stdin

# Verificar la sesión
podman login --get-login docker.io
```

```
# Dónde quedaron las credenciales (rootless)
ls -l "${XDG_RUNTIME_DIR}/containers/auth.json"
unset DH_TOKEN
```

> **Las credenciales no son permanentes en rootless**
>
> `${XDG_RUNTIME_DIR}` vive en `/run/user/$UID` y se borra al cerrar la última sesión del usuario, salvo que tengas `loginctl enable-linger` activo. Para que un servicio automatizado conserve el login, usa `--authfile` apuntando a una ruta persistente: `podman login --authfile ~/.config/containers/auth.json docker.io …`

3. La aplicación a publicar

Un servicio en Python puro, sin dependencias externas, que expone su propia versión y metadatos. Así puedes comprobar desde el navegador qué versión quedó publicada.

```
bitacora-api/
├── Containerfile
├── app.py
└── .dockerignore
```

app.py Copiar

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import json
import os
import platform
import socket

VERSION = os.environ.get("APP_VERSION", "desconocida")
COMMIT = os.environ.get("APP_COMMIT", "local")


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            datos = {"status": "ok"}
        else:
            datos = {
                "servicio": "bitacora-api",
                "version": VERSION,
                "commit": COMMIT,
                "host": socket.gethostname(),
                "arquitectura": platform.machine(),
```

```
                "python": platform.python_version(),
            }

        cuerpo = json.dumps(datos, indent=2, ensure_ascii=False).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(cuerpo)))
        self.end_headers()
        self.wfile.write(cuerpo)

    def log_message(self, fmt, *args):
        print(fmt % args, flush=True)


if __name__ == "__main__":
    puerto = int(os.environ.get("PORT", 8080))
    print(f"bitacora-api {VERSION} escuchando en 0.0.0.0:{puerto}", flush=True)
    HTTPServer(("0.0.0.0", puerto), Handler).serve_forever()
```

El Containerfile usa **ARG** para inyectar versión y commit al construir, y etiquetas OCI estándar que Docker Hub y las herramientas de inventario saben leer.

Containerfile

Copiar

```
FROM registry.access.redhat.com/ubi9/python-311

# Valores que se inyectan al construir
ARG APP_VERSION=0.0.0
ARG APP_COMMIT=local
ARG BUILD_DATE

# Etiquetas OCI estandar: las lee Docker Hub, skopeo y los escaneres
LABEL org.opencontainers.image.title="bitacora-api" \
      org.opencontainers.image.description="API de bitacora en Python puro" \
      org.opencontainers.image.version="${APP_VERSION}" \
      org.opencontainers.image.revision="${APP_COMMIT}" \
      org.opencontainers.image.created="${BUILD_DATE}" \
      org.opencontainers.image.authors="guerrero11dan" \
      org.opencontainers.image.source="https://github.com/dguerrero11/bitacora-api" \
      org.opencontainers.image.licenses="MIT"

WORKDIR /opt/app-root/src
COPY --chown=1001:0 app.py .

ENV APP_VERSION=${APP_VERSION} \
    APP_COMMIT=${APP_COMMIT} \
    PORT=8080 \
    PYTHONUNBUFFERED=1

EXPOSE 8080
```

```
USER 1001

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/health')

CMD ["python", "app.py"]
```

.dockerignore

Copiar

```
.git
.github
*.md
__pycache__/
*.pyc
.env
*.log
```

4. Construir con metadatos

Los valores dinámicos entran por `--build-arg`. Fíjate en el formato de fecha: RFC 3339 es lo que exige la especificación OCI.

terminal

Copiar

```
cd bitacora-api

VERSION=1.0.0
COMMIT=$(git rev-parse --short HEAD 2>/dev/null || echo manual)
FECHA=$(date -u +'%Y-%m-%dT%H:%M:%SZ')

podman build \
  --build-arg APP_VERSION="$VERSION" \
  --build-arg APP_COMMIT="$COMMIT" \
  --build-arg BUILD_DATE="$FECHA" \
  -t bitacora-api:"$VERSION" .

# Probar antes de publicar: nunca se sube algo que no arrancó
podman run -d --name prueba -p 8080:8080 bitacora-api:"$VERSION"
curl -s http://localhost:8080 ; echo
curl -s http://localhost:8080/health ; echo
podman rm -f prueba

# Revisar las etiquetas que quedaron grabadas
podman inspect bitacora-api:"$VERSION" \
  --format '{{json .Labels}}' | python3 -m json.tool
```

5. Etiquetar para Docker Hub

Un mismo contenido lleva varios nombres. Todos apuntan al mismo digest, así que etiquetar no duplica nada en disco ni en el registro.

La convención más usada es **versionado semántico en cascada**: quien fija `1.0.0` obtiene siempre lo mismo; quien sigue `1.0` recibe parches; quien sigue `1` recibe funciones nuevas compatibles.

terminal Copiar

```
REPO=docker.io/guerrero11dan/bitacora-api

podman tag bitacora-api:"$VERSION" "$REPO":"$VERSION"   # 1.0.0
podman tag bitacora-api:"$VERSION" "$REPO":1.0
podman tag bitacora-api:"$VERSION" "$REPO":1
podman tag bitacora-api:"$VERSION" "$REPO":latest

podman images --filter reference="$REPO"
```

> **latest no significa lo más reciente**
>
> Es solo el tag que Podman asume cuando no especificas ninguno. Si publicas `2.0.0` y olvidas mover `latest`, quien haga `pull` sin tag seguirá recibiendo la versión vieja. En producción nunca se despliega `latest`: el despliegue es irreproducible y no permite rollback confiable.

6. Publicar

Cada tag se envía por separado, pero las capas solo suben una vez: los tags siguientes terminan en segundos.

terminal Copiar

```
# El primer push sube las capas; los demás solo crean referencias
podman push "$REPO":"$VERSION"
podman push "$REPO":1.0
podman push "$REPO":1
podman push "$REPO":latest

# En una sola línea para todos los tags
for t in "$VERSION" 1.0 1 latest; do podman push "$REPO":"$t"; done
```

Errores comunes y qué significan

Mensaje	Causa	Solución
`unauthorized: authentication required`	Sin login, o el token expiró o se revocó	Repetir `podman login`
`requested access to the resource is denied`	El namespace no coincide con tu usuario	El tag debe empezar con `guerrero11dan/`
`denied: insufficient scopes`	Token con permiso de solo lectura	Generar uno con Read & Write
`toomanyrequests`	Límite de tasa de Docker Hub	Esperar, o autenticarse también para los pulls
`short-name did not resolve`	Falta el registro en el nombre	Usar siempre `docker.io/…` completo

7. Verificar la publicación

No basta con que el push termine sin error. Comprueba desde fuera, sin usar tu caché local.

terminal — Copiar

```
# A) Inspeccionar el remoto sin descargar nada (skopeo viene en RHEL)
skopeo inspect docker://"$REPO":1.0.0

# B) Listar todos los tags publicados
skopeo list-tags docker://"$REPO"

# C) Comparar digests: el local y el remoto deben coincidir
podman inspect "$REPO":1.0.0 --format '{{index .RepoDigests 0}}'
skopeo inspect docker://"$REPO":1.0.0 | grep -i digest

# D) Prueba real: borrar lo local y descargar del registro
podman rmi -f "$REPO":1.0.0 bitacora-api:1.0.0
podman pull "$REPO":1.0.0
podman run --rm -p 8080:8080 -d --name verif "$REPO":1.0.0
curl -s http://localhost:8080 ; echo
podman rm -f verif

# E) Desplegar por digest, que es lo reproducible
DIGEST=$(skopeo inspect docker://"$REPO":1.0.0 --format '{{.Digest}}')
podman run --rm "$REPO"@"$DIGEST" python -c "print('digest fijo, contenido garantizado')"
```

En la página web del repositorio, `hub.docker.com/r/guerrero11dan/bitacora-api` , agrega una descripción corta y un README. Docker Hub no lo toma del Containerfile: hay que escribirlo en la pestaña de descripción o sincronizarlo desde CI.

8. Imagen multiarquitectura

Si alguien con una Mac de Apple Silicon o un servidor ARM hace pull de tu imagen amd64, arranca con emulación lenta o no arranca. La solución es un manifest list: un índice que apunta a una imagen por arquitectura.

```
manifest list :1.0.0
      ├── linux/amd64  → sha256:aaa…
      └── linux/arm64  → sha256:bbb…

El cliente pide :1.0.0 y el registro le entrega
la variante que corresponde a su plataforma.
```

terminal Copiar

```
# Emulación de otras arquitecturas (una sola vez por host)
sudo dnf install -y qemu-user-static
sudo systemctl restart systemd-binfmt

# Construir las dos variantes bajo un mismo manifest
podman build \
  --platform linux/amd64,linux/arm64 \
  --manifest "$REPO":"$VERSION" \
  --build-arg APP_VERSION="$VERSION" \
  --build-arg APP_COMMIT="$COMMIT" \
  --build-arg BUILD_DATE="$FECHA" \
  .

podman manifest inspect "$REPO":"$VERSION"

# Publicar el índice completo: ojo, es manifest push, no push
podman manifest push --all "$REPO":"$VERSION" docker://"$REPO":"$VERSION"

# Confirmar que el registro devuelve las dos plataformas
skopeo inspect --raw docker://"$REPO":"$VERSION" | python3 -m json.tool | grep -A2 architectur
```

Firmar la imagen con Cosign

La firma permite que quien la consuma verifique que salió de ti y no fue alterada en el registro.

terminal

Copiar

```
cosign generate-key-pair              # genera cosign.key y cosign.pub
cosign sign --key cosign.key "$REPO":"$VERSION"
cosign verify --key cosign.pub "$REPO":"$VERSION"
```

> **La llave privada no se versiona**
>
> Agrega `cosign.key` a `.gitignore`. En CI va como secreto del repositorio, nunca en el árbol de archivos.

9. Publicación automatizada

El paso natural del ejercicio: que cada etiqueta de Git produzca y publique una imagen sin que nadie toque la terminal.

En GitHub, crea los secretos `DOCKERHUB_USER` con valor `guerrero11dan` y `DOCKERHUB_TOKEN` con el token del paso 2, en Settings → Secrets and variables → Actions.

.github/workflows/publicar.yml

Copiar

```
name: Publicar en Docker Hub

on:
  push:
    tags: ['v*.*.*']
  workflow_dispatch:

jobs:
  publicar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Calcular version
        id: meta
        run: |
          echo "version=${GITHUB_REF_NAME#v}" >> "$GITHUB_OUTPUT"
          echo "fecha=$(date -u +'%Y-%m-%dT%H:%M:%SZ')" >> "$GITHUB_OUTPUT"

      - name: Construir con Buildah
        id: build
        uses: redhat-actions/buildah-build@v2
```

```
        with:
          image: bitacora-api
          tags: ${{ steps.meta.outputs.version }} latest
          platforms: linux/amd64,linux/arm64
          containerfiles: ./Containerfile
          build-args: |
            APP_VERSION=${{ steps.meta.outputs.version }}
            APP_COMMIT=${{ github.sha }}
            BUILD_DATE=${{ steps.meta.outputs.fecha }}

      - name: Publicar
        uses: redhat-actions/push-to-registry@v2
        with:
          image: ${{ steps.build.outputs.image }}
          tags: ${{ steps.build.outputs.tags }}
          registry: docker.io/guerrero11dan
          username: ${{ secrets.DOCKERHUB_USER }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
```

terminal Copiar

```
git tag -a v1.1.0 -m "Agrega endpoint de metadatos"
git push origin v1.1.0
# El workflow construye, etiqueta y publica solo
```

10. Retos y buenas prácticas

Reto 1: reducir el tamaño con una construcción multietapa

Compila o instala dependencias en una etapa y copia solo el resultado a una imagen mínima. Compara con `podman images` antes y después.

Containerfile.multietapa Copiar

```
FROM registry.access.redhat.com/ubi9/python-311 AS builder
WORKDIR /build
COPY app.py .
RUN python -m compileall -q app.py

FROM registry.access.redhat.com/ubi9/python-311-minimal
WORKDIR /opt/app-root/src
COPY --from=builder --chown=1001:0 /build/app.py .
ENV PORT=8080 PYTHONUNBUFFERED=1
EXPOSE 8080
USER 1001
```

```
CMD ["python", "app.py"]
```

Reto 2: escanear antes de publicar

terminal Copiar

```
podman run --rm -v /var/run:/var/run:Z \
  docker.io/aquasec/trivy:latest image "$REPO":1.0.0

# Convertir hallazgos críticos en un fallo de build
trivy image --exit-code 1 --severity CRITICAL "$REPO":1.0.0
```

Reto 3: copiar entre registros sin descargar

Publica la misma imagen en Quay.io con skopeo, que transfiere de registro a registro sin pasar por tu disco. Es lo que se usa para promover imágenes entre entornos.

terminal Copiar

```
podman login quay.io
skopeo copy --all \
  docker://docker.io/guerrero11dan/bitacora-api:1.0.0 \
  docker://quay.io/guerrero11dan/bitacora-api:1.0.0
```

Reto 4: cerrar el ciclo con el pod anterior

Modifica el `lab-api.container` del ejercicio de pods para que use `docker.io/guerrero11dan/bitacora-api:1.0.0` en lugar de la imagen local, agrega `AutoUpdate=registry` en la sección `[Container]` y habilita el temporizador. Publica `1.0.1` y observa cómo el pod se actualiza solo.

terminal Copiar

```
systemctl --user enable --now podman-auto-update.timer
podman auto-update --dry-run
podman auto-update
```

Lista de verificación antes de cada publicación

- La imagen arrancó y respondió localmente.
- No corre como root: hay una instrucción `USER` .

- El tag es una versión concreta, no solo `latest`.
- Las etiquetas OCI llevan versión, revisión y fecha.
- No hay secretos en el Containerfile ni en variables de entorno de la imagen.
- El `.dockerignore` excluye `.git` y archivos de configuración local.
- Pasó el escaneo de vulnerabilidades críticas.

Limpieza

terminal

Copiar

```
podman logout docker.io
podman rmi -f "$REPO":1.0.0 "$REPO":1.0 "$REPO":1 "$REPO":latest
podman image prune -f
# Borrar el repositorio remoto se hace desde hub.docker.com, en Settings
```